FEATURE DEMO · FIX-ME OVERLAY

See what's overflowing, not just that it is.

The existing red "Overflows" ring names the slide. This adds a yellow tag that names the specific element responsible — "Fix Me" when the cause can be proven, "Likely fix" when it's the best available guess.

DECISION REQUIRED

A deliberately over-stuffed comparison.

This slide exists to demonstrate the Fix-Me overlay — open it in a live preview (Playground, Drawing Board, Studio, or the VS Code marp preview) to see the red ring and the yellow tag land on "Preferred option" alone, not "Alternative option" or the whole panel. A static export shows neither: the export strips both markers and clips the content instead, so this page prints a clean but visibly cut-off card.

ALTERNATIVE OPTION

- A short first fact about the alternative
- A short second fact about the alternative

PREFERRED OPTION

- This option's second bullet is written long on purpose, padding well past what the panel comfortably holds so the bounded content cell genuinely clips it — the exact case the overlay is built to catch, padding padding padding padding padding padding padding padding padding.
- A third fact, still padding this option out further so the compare-right cell overflows for real, not just in theory, padding padding padding padding padding padding

Which card is the actual problem?

Short card

One short line.

Short card two

Another short line here.

The oversized card

This card's body is written deliberately long, forcing its row to grow
far taller than its neighbor and threatening to push the whole grid
past the frame, padding padding padding padding padding padding padding
padding padding padding padding padding padding

Short card four

The last short line — stretched to match its tall row-mate, but not
the cause.

Two signals, tiered by confidence — never a guess dressed as a fact.

A bounded content cell (`.cell-stage`, `.panel-right`, `.compare-right`) that overflows genuinely clipped its own content — it never pushed a neighbor, so highlighting it is a geometric fact. Where the cell holds a repeated collection, the tag narrows to whichever item is a real content outlier.

CASE B · NO CLIP-CELL IN PLAY

Which milestone actually blew the budget?

`timeline-list` *is never wrapped in a bounded cell, so a clip-cell probe finds nothing here even though this slide genuinely overflows — the exact gap Case B closes. Open this in a live preview: the yellow tag reads "Likely fix," not "Fix Me," because a word count past budget is the best available guess, never a geometric certainty.*

---

changed here.

demanded their own pet feature get bolted onto the roadmap, and nobody on the steering committee was willing to say no to

budget.

best.

No clip-cell at all is now covered, hedged honestly.

A slide with no bounded cell falls back to the component's own word budget: whichever item runs furthest past `density.hard` is the best content-grounded guess. The copy says so — Case A reads "Fix Me," Case B reads "Likely fix." An oversized image, wide table, or long code block has no signal to drill into; the red ring alone fires there.

Preview-only. Never in the deliverable.

The overlay lives entirely in the runtime script every live preview loads — never in the export pipeline — so a shipped PDF, PPTX, or HTML export is byte-identical whether a slide overflowed during authoring or not.